IS212 (AY 2026/27, T1) – Scrum Project Instructions

Executive Summary

Overall Goal	Deliver a quality software increment for a customer using scrum
Weightage	35% of all course assessments

Deliverables & Due Dates

Scrum Process Consultation (ungraded)	During the Week 7 class: ● Each team will have an informal (ungraded) conversation with their instructors about how they are conducting their sprints. ● We recommend that teams start sprinting in Week 4 to get their 'hands dirty'. Figure out what is working and what is not, then use the Week 7 conversation to improve your process for the final sprint(s).
Software Release & Scrum Deliverables (graded)	By 23:59pm on Friday Week 12 (all sections): ● Submit a working software release based on the customer briefing that implements the core features. ● Submit all evidence of your scrum process and all code, including recordings of your final sprint meetings. ● Submissions must be made through eLearn; late penalties apply.
Interactive Q&A Session with Instructors (graded)	During your Week 13 class: ● Your team will meet your instructors for a closed-door up to 20-minute interactive Q&A session (i.e. no formal presentation). ● Your instructors will ask questions about your scrum process and software increment. ● You are encouraged to show evidence from your deliverables when answering your instructors' questions.
Peer Review (graded)	By Friday Week 14, 23:59pm: ● Complete your intra-team peer evaluation. ● Non-participation in the project / peer review will result in penalties.

Project Goal

The overall goal of this project is to give you hands-on experience of running an agile software project. It is designed to give you a sense of how real-world software developers collaborate both within a team and with an external customer (the good, the bad, and the ugly).

Based on the customer briefing provided to you in Week 1, and the subsequent clarification sessions, you are to nurture a product backlog of user stories, design a software system, and ultimately build a first release that increments all core features (specified below).

You are to manage your coding activities using the scrum process taught in class and are to document evidence of your various scrum activities and meetings throughout the entire project. Unlike other SCIS projects, our focus is on the process you are following, so please do not hack a system together in Week 12 and expect to score well.

We want to see a sustainable scrum process that is calibrated to the velocity of your team's developers. For example, it would be reasonable to conduct four 2-week sprints. We strongly encourage teams to utilise AI for code generation, but do remember that you are accountable for everything that you submit.

Your final sprint must be scheduled/planned to end by Friday Week 12 at the latest. We also want to see evidence of the steps you have taken to ensure the quality of your code, such as test case design, unit/integration testing, continuous integration, and/or refactoring. If your team starts sprinting before these topics are taught, it is fine to incorporate them into later sprint(s) when your process should be at its most refined.

First Release – Core Functionality

The customer briefing describes an entire system that is beyond the scope of a single-term project. From the functionality described in the Week 1 Customer Briefing, the customer has identified the following 20 features as the core requirements for the first release (i.e. your submission in Week 12). In particular:

Functionality Area	**Description**
User Authorisation and Authentication	Allows users to securely access the system. Access to information and functionality depends on whether the user is an Event Organiser, Event Coordinator, Venue Staff member, Technical Support Staff member, or Attendee. Users should only be able to view or modify information appropriate to their role and relationship to an event.
Event Request Creation	Event Organisers can create and submit event requests containing the information needed by ConnectSphere, including the event name, purpose, description, proposed date and time, expected attendance, venue requirements, accessibility needs, equipment requirements, and registration needs where relevant.
Draft Event Requests	Event Organisers can save an incomplete event request before submission, return to it later, and continue editing it. Draft requests should remain distinguishable from requests that have already been submitted for review.

Functionality Area	Description
Event Review and Approval	Event Coordinators can review submitted event requests, request clarification or amendments from the Event Organiser, and approve or reject requests where appropriate. The outcome of the review should be visible to the relevant users.
Coordinator Assignment	Submitted events can be assigned to an Event Coordinator who becomes the main internal point of contact responsible for coordinating the event. Events may subsequently be reassigned when staff responsibilities or availability change.
Event Status Management	Events progress through suitable statuses so that relevant users can understand their current stage. Examples may include draft, submitted, under review, approved, planning, confirmed, completed, cancelled, or rejected. Status changes should be consistent with the actions performed on the event.
Event Information Management	Authorised users can view and update relevant event information during the planning process. The system should distinguish between information that may be edited normally and important changes that may affect arrangements already made for the event.
Venue Catalogue	The system maintains information about ConnectSphere venues, including location, capacity, facilities, accessibility, supported room layouts, operating information, and other characteristics needed by Event Coordinators and Venue Staff when planning events.
Venue Availability Calendar	Authorised internal users can view venue availability across relevant dates and times. Existing bookings and other recorded periods of unavailability should be reflected so that users can understand when a venue may be requested.
Venue Search and Filtering	Event Coordinators can search and filter potential venues using relevant event requirements such as date, time, expected attendance, location, capacity, accessibility, supported layout, and required facilities.

Functionality Area	Description
Venue Suitability Checking	The system should assist users in determining whether a venue appears suitable for an event based on available event and venue information. For example, a venue should not normally be treated as suitable when the expected attendance exceeds its capacity or a required facility is unavailable.
Venue Booking Request	An Event Coordinator can submit a request to book a venue for an event. The request should contain the relevant event timing and venue requirements needed by Venue Staff to assess the booking.
Venue Booking Approval	Venue Staff can review pending venue booking requests and approve or reject them. Where a booking cannot be accepted, Venue Staff should be able to provide relevant information or a reason so that the Event Coordinator can continue planning.
Booking Conflict Detection	The system should identify incompatible or overlapping venue bookings and help prevent inappropriate double-booking. A confirmed booking should affect whether that venue is considered available for other events during the corresponding period.
Equipment Request Management	Event Coordinators can record equipment required for an event, including the equipment type, quantity, and relevant technical requirements. Technical Support Staff can review the requested equipment and update the request as arrangements are made.
Equipment Availability Checking	Technical Support Staff can determine whether sufficient suitable equipment is available for an event at the required date and time. Equipment already committed to another incompatible event or recorded as unavailable should not be treated as freely available.
Equipment Reservation	Available equipment can be reserved for an event. A reservation should reduce the quantity considered available for other overlapping events and should remain associated with the event for which it was reserved.
Attendee Registration	Where attendee registration is enabled, Attendees can view appropriate event information, register for an event, view their registration status, and withdraw their registration. Event Organisers and Event Coordinators can view appropriate registration information for the events they manage.

Functionality Area	Description
Event Change Requests	Event Organisers can request permitted changes after an event has been submitted. Event Coordinators can review and process the request. Changes to important information such as date, time, expected attendance, venue requirements, or equipment requirements may require existing arrangements to be reconsidered.
Notification System	Users can receive notifications about significant events relevant to them, such as event submission, clarification requests, approval or rejection, coordinator assignment, venue booking decisions, important event changes, registration updates, confirmation, or cancellation where applicable.

These core features should not be treated as isolated functionality. Your software release should support coherent end-to-end event workflows across the relevant features. For example, an Event Organiser's request should progress through review and venue booking, while venue and equipment reservations should reflect the date and time of the corresponding event. Changes to an event may also affect bookings, equipment, attendee registration, or notifications. Your design, implementation, and tests should reflect these relationships where they are relevant to the customer requirements.

Note that even though the Week 12 release only requires working code for these core features, you may still want to consider features beyond the scope of the first release in your product backlog and C4 models.

Deliverables – Working Code & Evidence of Scrum

It is important that you start collecting evidence of your team's sprinting process from the first sprint onwards, as this forms a significant part of your Week 12 submission. The quality of your scrum process (and thus the evidence you collect) weighs heavily when we grade your projects.

We encourage you to decide as a team how to manage your scrum project. You could, for example, manage your product and sprint backlogs using spreadsheets. Alternatively, you could explore tools such as Jira. The specific process you put together will determine the type of evidence you need to collect, e.g., PDF exports of spreadsheets vs. exports/screenshots from Jira.

Note that it is fine for your initial sprint to be a bit 'rough', as long as you show evidence of reflecting on this and taking steps to improve your subsequent sprints. We will view your final sprint as a "showcase sprint" that demonstrates your process at its best.

Submit one .zip file organised into the following numbered sub-folders:

#	Deliverable	Type
1	Your full, final product backlog, containing all your completed / pending user stories (if you use a tool like Jira, you need to export your backlog/stories for the submission; we will not be accessing your Jira project directly).	Document
2	Your system design, conveyed using the C4 model.	Document(s)

3	Test cases covering all core features. Your test evidence should make it possible to trace important user stories or acceptance criteria to the relevant tests. Include appropriate unit, integration, and/or end-to-end tests where relevant.	Document(s)
#	**Deliverable**	**Type**
4	Documents / screenshots from your sprint meetings (all sprints), for example: sprint backlogs/goals/tasks at the start of each sprint; records of estimation activities; scrum task boards at the end of each sprint; burndown charts at the end of each sprint; and other relevant meeting documents such as sprint retrospective boards. IMPORTANT: for your final sprint only, additionally include recordings of your sprint planning, review, retro, and one daily standup. Upload large recordings as unlisted videos and include links in your submission. Do not include the videos themselves in your submission. Note: Zoom/Teams AI transcripts are great at keeping minutes!	Documents & Videos
5	A self-contained slide deck summarising how your team addressed the rubrics of this course project. (Note that this will not be presented.) IMPORTANT: this must follow the slide template we will release in eLearn by Week 11.	PDF Slide Deck
6	Your software release that includes all core features. Include all working code; a README with clear instructions on how to run your software release; relevant test classes containing unit/integration tests; and any CI pipeline script that you utilised.	Code
7	A README file that contains a link to your Git repository; you must also ensure that both of your instructors have been granted access to the repository.	README & Git Repository Access

Week 13 Interactive Q&A

- Your team will meet its instructors for a closed-door up to 20-minute interactive Q&A session. There is no formal presentation.
- Instructors may ask you to demonstrate selected functionality, explain important design decisions, trace a requirement to its implementation and tests, or discuss how your team responded to changes and feedback across sprints.
- You are encouraged to show evidence from your submitted deliverables when answering.

eLearn Submission Requirements & Late Penalties

1. Submit all deliverables as a single .zip file organised into numbered sub-folders 1-7 according to the table above.
2. Submit to your section's project assignment on eLearn by 23:59 on Friday Week 12.
3. An intra-team peer review will be conducted in Week 14. Students deemed not to have contributed may receive a penalty after investigation. Inform the teaching team early if serious team issues arise so that intervention is still possible.
4. Late submissions are subject to heavy penalties. Submit early. Only the latest submission will be graded.

Within 1 hour (one second late is late)	10% deduction from the marks deserved
Each subsequent hour	Penalty increases to 20%, 40%, 80%, and finally 100%

Project Rubrics

Projects will be graded holistically according to the rubric on the next page. The descriptors are indicative rather than a checklist. In particular, strong implementation cannot compensate fully for a weak or poorly evidenced scrum process. An AI-generated solution that you cannot explain/justify will not score well. Ultimately, remember that you are graded on your ability to satisfy the customer requirements (and those requirements may change).

Additional Guidance

- Base your scrum process on the version taught in class. Consult your instructors before adopting substantial variations found elsewhere.
- Do not post customer questions in public channels. Use only the clarification process communicated to your section.
- Each section may receive slightly different customer clarifications. Only your section's clarifications apply to your team.

	Unsatisfactory	Fair	Good	Excellent
Scrum process and evidence	Work is largely ad hoc, with little evidence of sustained sprinting, inspection, or adaptation.	Some scrum activities are performed, but inconsistently or with limited documentation and improvement.	The team largely follows a coherent scrum process, with adequate evidence and visible improvement across sprints.	The team consistently follows an effective, sustainable scrum process supported by comprehensive evidence, reflection, and adaptation.
Estimation and planning	Estimation techniques taught in class are absent or estimates are not used meaningfully.	Estimation is used arbitrarily or inconsistently, with weak justification or connection to capacity and velocity.	Estimation is applied consistently with adequate justification and is used to plan work against team capacity.	Estimation is consistent, well-justified, and refined using evidence to produce valid and reasonable plans calibrated to team velocity.
User stories and customer needs	Stories are unclear, poorly structured, or disconnected from customer needs and acceptance criteria.	Stories broadly follow the prescribed format but require significant clarification and have weak acceptance criteria.	Stories follow the prescribed format, reflect user needs, and contain generally clear acceptance criteria.	Stories clearly demonstrate user needs, conversations, constraints, and testable acceptance criteria, including appropriate handling of important edge cases and changed requirements.
System design	The system is poorly modelled or significant design responsibilities and decisions are unclear.	Some design is documented, but significant decisions are incomplete, overly rigid, or weakly justified.	The system is well modelled, showing significant design decisions and suitable separation of responsibilities.	The design clearly communicates significant decisions, good use of design principles and reusable patterns, and well-justified trade-offs that support change.

	Unsatisfactory	Fair	Good	Excellent
Working software	The system barely functions or major core features are absent.	Some features work, but important workflows are incomplete, fragile, or require substantial manual workarounds.	The core features work reasonably well and important end-to-end workflows can be demonstrated with minor issues.	The solution is modular, reliable, and demonstrable; the core workflows behave consistently, including important conflict, change, and failure scenarios.
Testing and traceability	There is minimal testing or little connection between requirements, code, and tests.	Some tests exist but coverage, traceability, or failure-case testing is limited.	Core features are covered by traceable test cases and meaningful code-level unit and/or integration tests.	Testing is comprehensive and traceable, combining appropriate unit and integration/end-to-end tests for normal, boundary, conflict, and failure scenarios. 100% coverage unless not possible and stated why.
Code quality and CI	Code is poorly organised and there is minimal evidence of quality practices or automated integration.	Some structure and quality practices are visible, but refactoring or CI is limited or applied late.	Code is reasonably modular, with evidence of refactoring and a basic CI pipeline used during development.	Code is modular and maintainable, quality practices are sustained, and a comprehensive CI pipeline reliably builds and tests the system.
Accountability and Explainability	Unable to explain why things are designed or implemented as they are; relies on unverified or AI-generated code without understanding.	Demonstrates basic awareness of code and design, but struggles to justify specific choices, trade-offs, or AI-generated code when questioned.	Explains most design decisions, code implementations, and scrum processes clearly; demonstrates solid ownership of all deliverables.	Demonstrates complete ownership and deep understanding of all code, architecture, and scrum decisions; articulately justifies trade-offs and AI contributions during Q&A.